

Deep Learning Sequential Models

Comprehensive Study Notes: LSTM and GRU Architectures

These notes cover the foundational neural network architectures designed for sequential data processing, extending your existing predictive modeling concepts into time-series and natural language applications.

1. Long Short-Term Memory (LSTM) Networks

1.1 Introduction to LSTMs

An LSTM is a specialized type of **Recurrent Neural Network (RNN)** designed to capture long-term dependencies in sequential data.

- **The Problem it Solves:** Traditional RNNs struggle with long-term dependencies due to the *vanishing gradient problem*. As sequences get longer, gradients used to update weights shrink exponentially, causing the network to "forget" earlier information.
- **The Solution:** LSTMs introduce a central *Cell State* and *Gates* that regulate information flow, selectively remembering important data and forgetting irrelevant data.

1.2 Core Architecture

Think of the LSTM as a conveyor belt moving through a sequence.

- **The Cell State (C_t):** The core memory. It runs down the entire chain with only minor linear interactions, allowing information to flow unchanged.
- **The Hidden State (h_t):** The working memory or output for the current time step.
- **Gates:** Neural network layers (using sigmoid activations) that output values between 0 and 1 to control information flow (0 = forget, 1 = remember).

1.3 The Step-by-Step Mathematics of an LSTM

An LSTM cell processes the current input (x_t) and previous hidden state (h_{t-1}) in four steps.

Step 1: The Forget Gate

Decides what information to throw away from the previous cell state.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Step 2: The Input Gate & Candidate Memory

Decides what new information to store in the cell state.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$C_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

Step 3: Updating the Cell State

We update the old cell state (C_{t-1}) into the new cell state (C_t).

$$C_t = f_t * C_{t-1} + i_t * C_t$$

Step 4: The Output Gate

We decide the final output based on a filtered version of our cell state.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

2. Gated Recurrent Units (GRU)

2.1 Introduction to GRUs

Introduced in 2014, the GRU is a streamlined variation of the LSTM. It solves the vanishing gradient problem with a simpler, more computationally efficient architecture.

- **The Main Difference:** GRUs combine the forget and input gates into a single *update gate* and merge the cell state and hidden state.
- **The Result:** GRUs have fewer parameters to train, making them faster and less memory-intensive, often achieving similar performance.

2.2 The Step-by-Step Mathematics of a GRU

Step 1: The Update Gate (z_t)

Determines how much past information to pass along to the future (acts like forget + input gates combined).

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$$

Step 2: The Reset Gate (r_t)

Decides how much past information to forget before computing the candidate state.

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$$

Step 3: The Candidate Hidden State ($\tilde{h}_t$)

Calculates new memory content. The reset gate is applied element-wise to the previous hidden state.

$$\tilde{h}_t = \tanh(W_h \cdot [r_t * h_{t-1}, x_t] + b_h)$$

Step 4: Final Hidden State (h_t)

Interpolates between the previous hidden state and the new candidate state using the update gate.

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

3. GRU vs. LSTM Comparison

Feature	GRU (Gated Recurrent Unit)	LSTM (Long Short-Term Memory)
Complexity	Simpler (fewer parameters)	More complex (more parameters)
Gates	2 Gates (Update, Reset)	3 Gates (Input, Forget, Output)
State Tracks	1 (Hidden State h_t only)	2 (Cell State C_t & Hidden State h_t)
Training Speed	Faster due to fewer tensor operations	Slower
Memory Usage	Lower	Higher
Best Use Case	Smaller datasets, or when computational power is limited.	Complex tasks with extremely long dependencies.